Real-Time Group Chat Application

Computer System Design (CSD) — Assignment 4 | Comprehensive Technical & Team Contribution Report

Course: Computer System Design (CSD)	**Assignment:** Assignment 4
Host SSH Machine: student@10.1.75.51 (Port 2237)	**Internal Port:** 5000
Official Testing Public URL: http://10.1.75.51:5237/	**Tech Stack:** Flask, Flask-SocketIO, SQLite, AES-256-GCM, Ed25519

1. Executive Summary & Lab Network Mapping

This project implements a multi-user, real-time Group Chat Application for **Computer System Design (CSD) Assignment 4**. Built using WebSockets (Flask-SocketIO) and SQLite persistence, it includes advanced security layers (AES-256-GCM confidentiality and Ed25519 digital signatures). The backend server runs on the allotted SSH laboratory machine (student@10.1.75.51).

Lab Network & Port Forwarding Mapping:
The Flask backend process binds internally to port 5000. The laboratory network NAT router maps SSH port 2237 to external web port 5237. Therefore, the official public URL for TA evaluation across all 4 student clients is: **http://10.1.75.51:5237/**.

2. Team Members & Key Technical Contributions

Role / Designation	Student Name	Roll No.	SSH Server	Key Technical Contribution
Group Head (Host)	Ranga Chandra Naga Venkata Chaitanya Kolla	12341740	ssh -p 2237	Backend Architecture, Flask-SocketIO Core, AES-256-GCM Encryp
Member 2	Bhukya Raju	12340520	ssh -p 2238	Front-End UI Design, Cyberpunk Aurora Glassmorphism Theme, Sc
Member 3	V.G.N. Harshitha	12342310	ssh -p 2239	Database Schema Architecture (chat.db), SQLite Message Persiste
Member 4	Maloth Madhu	12341370	ssh -p 2240	Multi-Client Integration & Testing across SSH Machines (10.1.75.51

3. Feature Coverage & Requirement Verification

Requirement / Feature	Implementation Mechanism	Status
Real-Time Message Broadcast	Bi-directional WebSocket event broadcasting via Flask-SocketIO	VERIFIED
User Join/Leave Alerts	Automatic broadcast of user arrival/departure pills to room	VERIFIED
User Identification	Unique username validation & dynamic gradient avatar generation	VERIFIED
Client Disconnection Cleanup	Server socket connection monitoring & cleanup on tab close	VERIFIED
SQLite Persistence	All messages saved in chat.db; auto-loaded upon joining chat	VERIFIED
Confidentiality (AES-256-GCM)	Payload encrypted with room key prior to database insertion	VERIFIED
Authenticity (Ed25519)	Cryptographic digital signature per user stored & verified	VERIFIED
Public TA Access URL	Mapped external port forwarded to http://10.1.75.51:5237/	VERIFIED

4. Application Screenshots & Visual Proof Placeholders

■ **Screenshot 1 Placeholder: Real-Time Chat Interface on http://10.1.75.51:5237/ (Multi-user chat & active sidebar)**
[Paste Screenshot Here]

■ **Screenshot 2 Placeholder: User Join Modal & Unique Username Validation Screen**
[Paste Screenshot Here]

■ Screenshot 3 Placeholder: SQLite Database Inspection Terminal (showing AES-256-GCM ciphertexts & Ed25519 signatures)
[Paste Screenshot Here]

■ Screenshot 4 Placeholder: SSH Host Background Daemon Process Running (nohup python3 server.py)
[Paste Screenshot Here]

5. Database Verification Commands (SSH Host)

To inspect encrypted messages and signatures in SQLite:
sqlite3 ~/chat_app/chat.db "SELECT id, sender, ciphertext, signature, timestamp FROM messages;"
To verify public Ed25519 user signing keys:
sqlite3 ~/chat_app/chat.db "SELECT username, public_key FROM signing_keys;"

6. Official Submission Details

- **GitHub Repository:** https://github.com/chaitanyakumarAI/Group-Chat
- **Live Application URL (For TA Testing): http://10.1.75.51:5237/**
- **Host SSH Server Command:** ssh -p 2237 student@10.1.75.51
- **Daemon Process Command:** nohup python3 server.py > server.log 2>&1 &